UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Materia

SOA

Manual Técnico

Librería Cartagena — tienda en línea

Arquitectura, despliegue y mantenimiento para el equipo de desarrollo.

Integrantes

Nombre	Número de cuenta
Sarahi Yamileth García Bojórquez	23020009
Aranzazú Acosta Medina	23021126
Alan Zayas Maldonado	23020112
Cesar Omar Karass Cota	23020143

Fecha de creación: 03 de noviembre de 2026

Documento académico — Materia SOA · Librería Cartagena

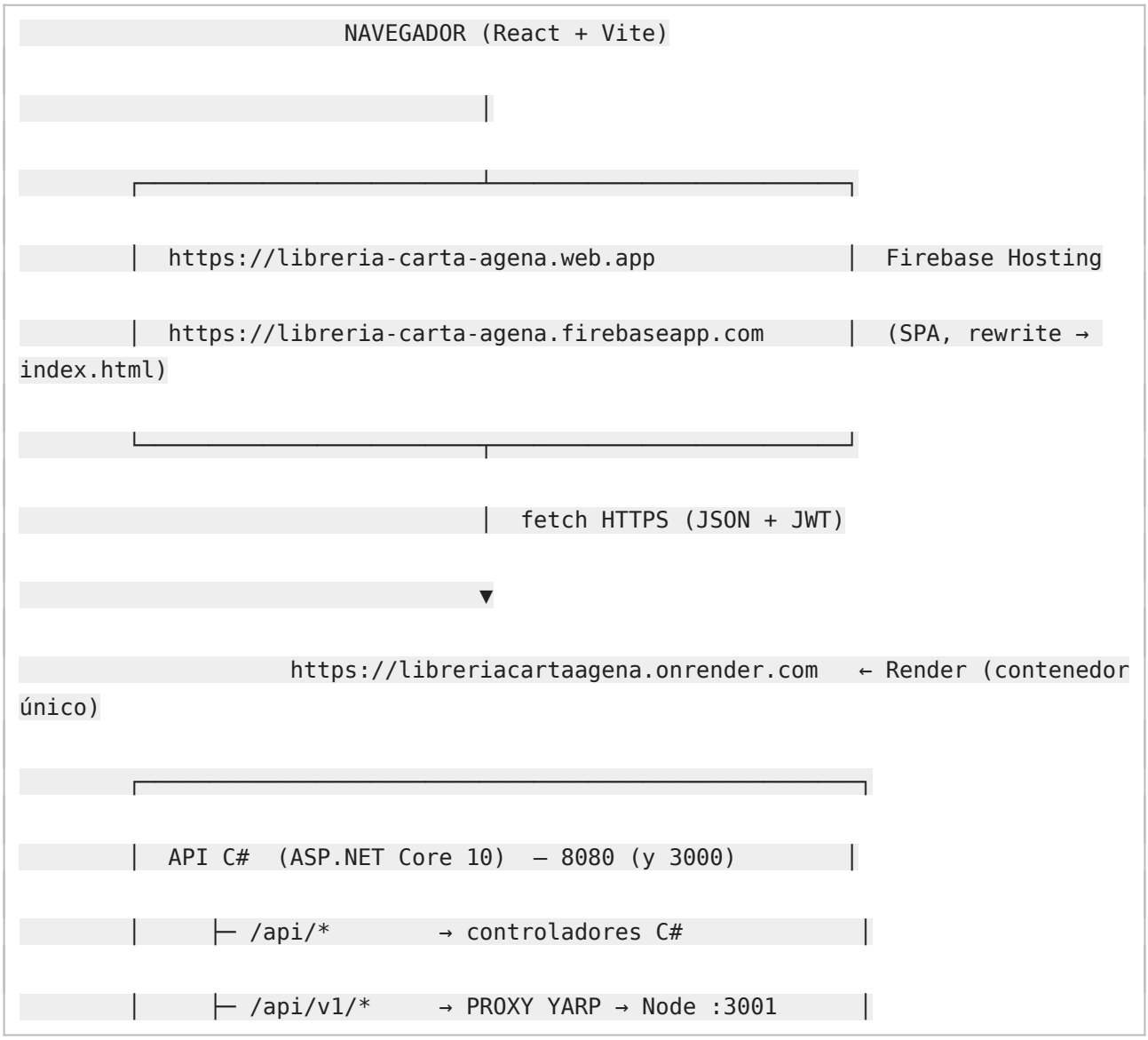
Manual Técnico — Librería Cartagena

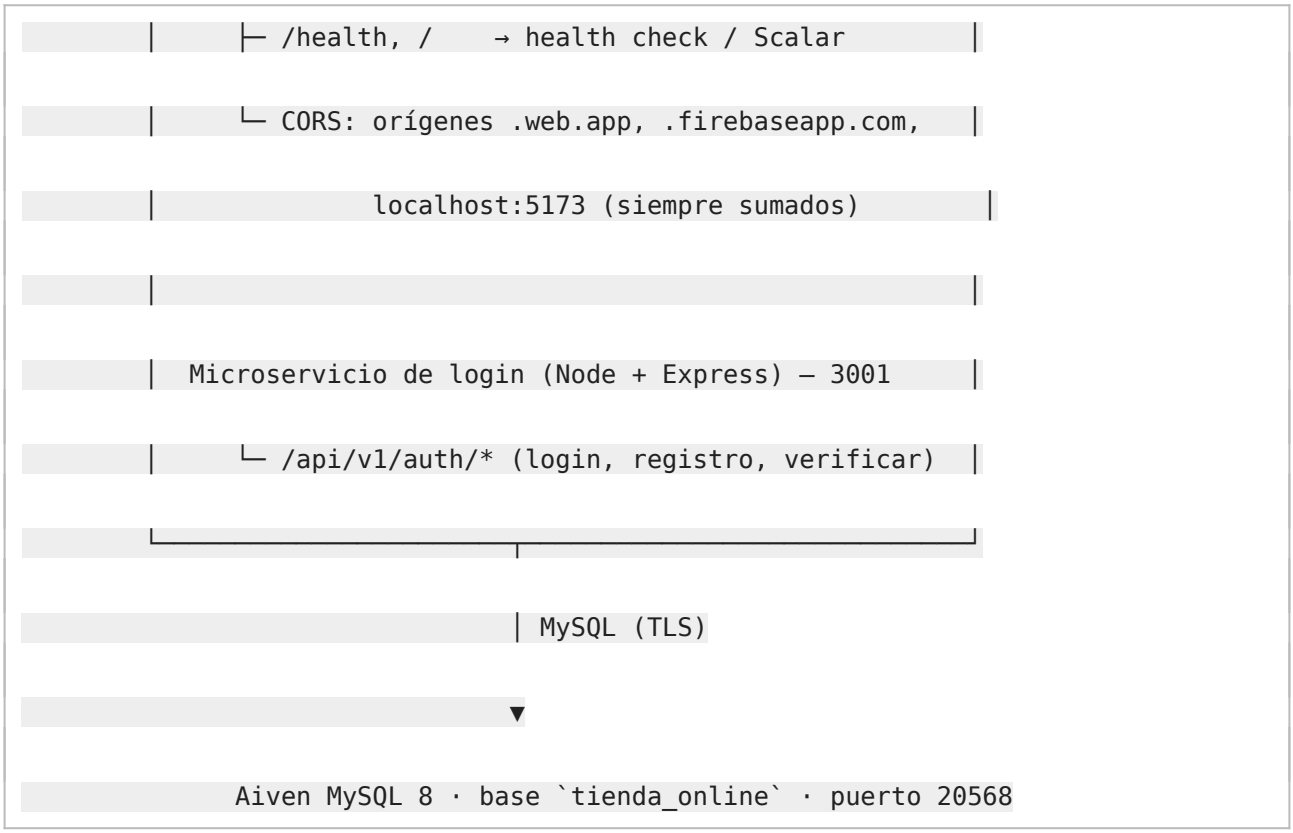
Documento para el equipo de desarrollo y mantenimiento. Describe la arquitectura, los repositorios, el modelo de datos, la seguridad, el envío de correos, los despliegues y la solución de problemas del sistema **Cartagena Bookstore**.

> Compañero de este documento: `docs/Manual-de-Usuario.md` (uso final).

---

1. Arquitectura





1.1 Topología

Entorno	Frontend	API C#	Login	Base de datos
Producción	Firebase Hosting	Render (`libreriacartaagena.onrender.com`)	**Dentro del mismo contenedor** (puerto interno 3001, publicado por el proxy YARP)	Aiven MySQL (`mysql-3da06998-…:20568`)
Local (nativo)	Vite `:5173`	`:5000`	`:3000`	`mysql-cartagena` en `127.0.0.1:3307`
Local (Docker)	Vite `:5173`	`:8083` (gateway `:8081`)	`:3000`	contenedor `mysql` en la red `libreria-network`

Regla de oro: el frontend nunca habla con MySQL; solo con HTTP (`/api/*` para la API C# y `/api/v1/*` para el login).

1.2 Estructura del repositorio

```
LibrereriaCartagena/

├── Frontend/                          React + Vite (SPA)

│   ├── src/

│   │   ├── routes/AppRoutes.tsx       rutas de la aplicación

│   │   ├── services/                  capa de acceso a la API (api.ts,
authService.ts, …)

│   │   ├── features/                  pantallas por dominio (auth, admin, …)

│   │   └── components/                componentes compartidos

│   ├── vite.config.ts                 proxy de desarrollo (ver §13.1)

│   └── .env.production                URLs de producción

├── Backend/Cartagena/

│   ├── backend/Cartagena.Api/         API C#

│   │   ├── Controllers/               14 controladores

│   │   ├── Services/                  MailService (correo) y PdfService (tickets)

│   │   ├── Infrastructure/            ApiException, manejador global, validaciones

│   │   ├── Data/DbFactory.cs          cadena de conexión + pool

│   │   ├── Program.cs                 CORS, políticas, JWT, proxy YARP

│   │   ├── Dockerfile                 imagen de producción (dotnet + Node)

│   │   └── entrypoint.sh              arranca login y API en el mismo contenedor
```

```
│   ├── servicios/microservicio-login/   microservicio Node (fuente)
│   ├── backups/tienda_online.sql   respaldo de las 19 tablas originales
│   └── docker-compose.yml          stack local completo
├── docs/                           manuales (fuente .md → .docx/.pdf) y capturas
├── PRUEBAS-ESTRUCTURALES.md        informe de pruebas
├── start-dev.sh / stop-dev.sh      stack local nativo
└── start-docker.sh / stop-local.sh stack local con Docker
```

---

2. Repositorios

Repositorio	Rama	Uso
`https://github.com/CesarKarass/LibreriaCartagena`	`Frontend` y `main`	Repositorio principal del equipo (colaboradores).
`https://github.com/Yami678/LibreriaCartaAgena`	`main`	**Desde aquí despliega Render** (copia sincronizada del backend).

El trabajo se hace en el repositorio principal y se sincroniza al de Render:

```
TOKEN=<con permisos de escritura en ambos>

# principal

git push
"https://x-access-token:${TOKEN}@github.com/CesarKarass/LibreriaCartagena.git"
Frontend:Frontend



# repositorio de Render (solo Backend/Cartagena)

rsync -a --exclude bin/ --exclude obj/ --exclude node_modules/ \
```

```
      --exclude backups/ --exclude .env \

      Backend/Cartagena/ /tmp/opencode/yami678/Backend/Cartagena/

cd /tmp/opencode/yami678 && git add -A && git commit -m "…" && \

  git push
"https://x-access-token:${TOKEN}@github.com/Yami678/LibreriaCartaAgena.git"
main:main
```

> **Nunca** sincronizar `.env`: está en `.gitignore` y contiene credenciales. Los repositorios son **públicos**: ninguna contraseña puede viajar en el código.

---

3. Componentes y puertos

Componente	Carpeta	Puerto	Notas
Frontend React + Vite	`Frontend/`	`5173` (dev)	Build estático a `dist/`.
API C# (ASP.NET Core 10)	`Backend/Cartagena/` `backend/Cartagena.Api/`	`5000` local · `8080` y `3000` en el contenedor	Dapper + MySqlConnector.
Microservicio de login	`Backend/Cartagena/` `servicios/microservicio-login/` (copia embebida en `Cartagena.Api/login/`)	`3000` local · `3001` en el contenedor	Express + JWT + bcrypt.
MySQL local	contenedor `mysql-cartagena`	`3307`	Base `tienda_online`, usuario `usuario_app`.
MySQL producción	Aiven	`20568`	Usuario `avnadmin`, TLS.

Flask de proveedores (legado)	`Backend/Cartagena/servicios/proveedores/`	`5000/5001`	**Ya no se usa**: su CRUD fue reimplementado en `ProveedoresController.cs`.

3.1 El contenedor de producción (único servicio)

`Dockerfile` de `Cartagena.Api`:

1. Etapa de build: .NET SDK 10 → `dotnet publish`.
2. Etapa final: aspnet 10 **+ Node.js 20** (descargado de nodejs.org).
3. Copia el microservicio a `/app/login` y ejecuta `npm ci --omit=dev`.
4. `ENTRYPOINT /entrypoint.sh`: - arranca el login Node en `PORT=3001` (proceso secundario); - `exec dotnet Cartagena.Api.dll` (proceso principal, recibe SIGTERM de Render).
5. `ENV ASPNETCORE_URLS=http://+:8080;http://+:3000` y **solo** `EXPOSE 8080`.

> ⚠️ **No exponer el 3000 como puerto del login.** Render enruta el tráfico al puerto que anuncia la imagen: si se publica el puerto del Node, *todo* el tráfico llega al Express y la API C# deja de responder (se perdían `/health`, `/api/*` y el proxy). El 3000 del contenedor es de la **API C#**, a modo de respaldo.

El proxy inverso está declarado en `Program.cs` con **YARP**:

```
Route: /api/v1/{**catch-all}  →  destino: LOGIN_PROXY_TARGET (http://localhost:3001)
```

> **Ojo en local:** `LOGIN_PROXY_TARGET` apunta al 3001 (el puerto del contenedor), pero en desarrollo nativo el login corre en el **3000**. Si no se define la variable, `curl localhost:5000/api/v1/...` responde `502` con *Connection refused (localhost:3001)*. El frontend no se ve afectado: en desarrollo Vite proxy `/api/v1` → `3000` directamente.

`login/LEEME.md` explica cómo volver a copiar el microservicio original si cambia.

---

4. Variables de entorno

4.1 API C# / contenedor (Render)

Variable	Ejemplo / valor en producción	Descripción

`DB_HOST`	`mysql-3da06998-…i.aivencloud.com`	Host MySQL.
`DB_PORT`	`20568`	Puerto MySQL.
`DB_NAME`	`tienda_online`	Base de datos.
`DB_USER`	`avnadmin`	Usuario MySQL.
`DB_PASSWORD`	*(secreto en Render)*	Contraseña MySQL.
`JWT_SECRET`	*(secreto, ≥ 32 bytes)*	Firma HS256. **Debe ser idéntico al del login.**
`JWT_ISSUER`	`microservicio-login`	Emisor del token.
`JWT_EXPIRES_IN`	`15m`	Vigencia del token (afecta la duración de sesión).
`CORS_ORIGIN`	`https://libreria-carta-agena.web.app,…`	Orígenes extra. Los dominios `.web.app` y `.firebaseapp.com` **se suman siempre** en código.
`LOGIN_PROXY_TARGET`	`http://localhost:3001`	Destino del proxy YARP.
`SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASSWORD` / `SMTP_FROM`	*(vacíos en Render)*	**Solo respaldo**: la configuración real vive en la tabla `config_sistema` (§8).
`RECAPTCHA_SECRET`	*(vacío)*	Verificación server-side del reCAPTCHA.

4.2 Microservicio de login (Node)

Variable	Valor por defecto	Descripción
`JWT_SECRET`	*(requerido)*	Misma firma que la API C#.
`JWT_EXPIRES_IN`	`15m`	Duración del token.
`BCRYPT_SALT_ROUNDS`	`12`	Coste del hash (4–15).

`LOGIN_RATE_LIMIT_WINDOW_MS`	`900000` (15 min)	Ventana del limitador de intentos de login.
`DB_*`	—	Misma base que la API C#.

4.3 Frontend (`Frontend/.env.production`)

Variable	Valor en producción
`VITE_API_URL`	`https://libreriacartaagena.onrender.com/api`
`VITE_AUTH_API_URL`	`https://libreriacartaagena.onrender.com/api/v1`
`VITE_RECAPTCHA_SITE_KEY`	site key de **prueba** de Google (ver §10)
`VITE_USE_MOCKS`	`false`

`.env.development` usa rutas relativas (`/api`, `/api/v1`) y el proxy de Vite.

---

5. API

Documentación interactiva: **`/scalar/v1`** (Scalar/OpenAPI) en el servidor.

5.1 Login (microservicio Node, vía proxy)

Método	Ruta	Descripción
POST	`/api/v1/auth/login`	Body `{"nombreUsuario","contrasena"}` → token JWT.
POST	`/api/v1/auth/registro`	Alta de cuenta (rol `CLIENTE` por defecto).
GET	`/api/v1/auth/usuario-disponible?nombreUsuario=`	Consulta de duplicados (registro).
GET	`/api/v1/auth/verificar`	Valida el token (`Authorization: Bearer`).
GET	`/api/v1/auth/perfil`	Perfil del usuario autenticado.
GET	`/api/v1/auth/admin/panel`	Confirma el rol ADMIN (panel).

GET	`/api/v1/estado`, `/api/v1/salud`	Estado del microservicio.

5.2 API C# (selección)

Método	Ruta	Rol
GET	`/api/libros`, `/api/libros/{id}`	público
GET/POST/ PUT/DELETE	`/api/proveedores`, `/api/proveedores/{id}`	ADMIN
GET/POST	`/api/compras-proveedor`, `…/{id}/recibir`	ADMIN
GET/POST	`/api/admin/inventario`, `/api/admin/usuarios`, `/api/admin/roles`	ADMIN
GET	`/api/admin/pedidos`, `/api/admin/resumen`, `/api/admin/logistica/envios`	ADMIN
POST	`/api/checkout`	autenticado
GET	`/api/mis-compras`, `/api/ventas/{id}/ticket.pdf`	autenticado
POST	`/api/cuenta/recuperacion/solicitar`, `…/restablecer`	público
GET	`/health`	público (health check de Render)
GET	`/api/core/correo/diagnostico`	público (diagnóstico SMTP, ver §9)

Convención de respuestas

- Éxito: `{ "success": true, "message"?: string, "data": … }` (snake_case).
- Error: `{ "error": "mensaje" }` con el código HTTP correspondiente (400 validación · 401 sin token · 403 rol insuficiente · 404 inexistente · 409 conflicto · 503 servicio caído o no configurado). El manejador global es `Infrastructure/ApiExceptionHandler.cs`.

5.3 Autorización

Políticas definidas en `Program.cs`:

Política	Roles aceptados

`Admin`	`ADMIN`
`Staff`	`ADMIN`, `REPARTIDOR`
`Comprador`	`CLIENTE`, `ADMIN`

El claim se lee de `rol` en el JWT emitido por el microservicio de login.

5.4 Módulo de proveedores (contrato)

`Controllers/ProveedoresController.cs` replica el contrato del antiguo Flask:

Método	Ruta	Éxito	Errores
GET	`/api/proveedores`	200 `{success,data}`	401/403
GET	`/api/proveedores/{id}`	200	404
POST	`/api/proveedores`	201 `{success,message,data}`	400 (faltan llaves/vacío), 409 (RFC duplicado)
PUT	`/api/proveedores/{id}`	200	400 (sin campos), 404, 409 (RFC de otro)
DELETE	`/api/proveedores/{id}`	200	404, 409 (tiene compras)

Reglas de validación idénticas al Flask original: en el **alta** se exigen las siete llaves (aunque el valor sea `null`), en la **edición** basta con un campo; `nombre` es obligatorio y no puede ir vacío; se respetan los largos de columna (150/150/150/13/255/150/50) para no provocar errores 1406.

---

6. Base de datos

Entorno	Conexión
Local	`127.0.0.1:3307`, base `tienda_online`, usuario `usuario_app`, contraseña en `Backend/Cartagena/.env`.

Producción	Aiven MySQL, `…i.aivencloud.com:20568`, usuario `avnadmin`, base `tienda_online`.

Son **20 tablas** y 135 columnas. Las principales:

Tabla	Columnas	Rol en el negocio
`rol`	`id_rol, nombre`	`ADMIN`, `CLIENTE`, `VENDEDOR`, `REPARTIDOR`.
`usuario`	`id_usuario, id_rol, nombre, correo, contrasena_hash, fecha_registro, nombre_usuario, telefono, correo_verificado, activo, intentos_fallidos, bloqueado_hasta, ultimo_acceso`	Cuenta y control de accesos.
`token_cuenta`	`id, id_usuario, tipo, hash_token, expira_en, usado_en`	Tokens de un solo uso (recuperación y verificación).
`libro`	`id_libro, titulo, autor, precio, stock, stock_minimo, stock_maximo, descripcion, isbn, editorial, categoria, genero, formato, imagen_portada, descuento_porcentaje, pdf_ruta`	Catálogo e inventario.
`carrito` / `item_carrito`	`id_carrito, id_usuario, total` / `id_item, id_carrito, id_libro, cantidad`	Carrito de compras.
`venta`	`id_venta, id_usuario, id_tipo_pago, fecha, total, estado, estado_pago, referencia_pago, clave_idempotencia`	Cabecera de la compra.
`detalle_venta`	`id_detalle, id_venta, id_libro, cantidad, precio_unitario`	Líneas de la compra.
`domicilio` / `envio` / `paquete`	—	Entrega y seguimiento.
`proveedor`	`id_proveedor, nombre, contacto, telefono, raz□on_social, rfc, correo, direccion`	Directorio de proveedores.
`pedido_compra` / `detalle_pedido_co`	`id_pedido, id_proveedor, fecha, costo_total, estado, fecha_recepcion`	Compras al

`mpra`		proveedor.
`devolucion_proveedor` / `movimiento_inventario`	—	Mermas, devoluciones y kardex.
`comentario` / `solicitud_garantia` / `tipo_pago`	—	Reseñas, garantías y métodos de pago.
`config_sistema`	`clave, valor, actualizado_en`	Ajustes modificables **sin redeploy** (§8).

6.1 Esquema de autenticación

El esquema de las tablas `rol`, `usuario` y `token_cuenta` vive en `servicios/microservicio-login/database/esquema.sql` y se aplica automáticamente al arrancar `start-dev.sh` (`CREATE TABLE IF NOT EXISTS`, es idempotente).

`config_sistema` se crea en el mismo archivo; en local queda vacía (manda el `.env`) y en producción se llena con las claves SMTP.

6.2 Sincronizar datos a Aiven (respaldo `tienda_online.sql`)

```
H=mysql-3da06998-….i.aivencloud.com

mysql -h $H -P 20568 -u avnadmin -p"$PW" --ssl-mode=REQUIRED \

  -e "DROP DATABASE IF EXISTS tienda_online; CREATE DATABASE tienda_online \

      CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;"

mysql -h $H -P 20568 -u avnadmin -p"$PW" --ssl-mode=REQUIRED \

  --init-command="SET FOREIGN_KEY_CHECKS=0" --default-character-set=utf8mb4 \

  tienda_online < tienda_online.sql
```

El `--init-command="SET FOREIGN_KEY_CHECKS=0"` es **obligatorio**: el dump ordena las tablas de forma que una clave foránea apunta a una columna aún no recreada (error `3780`).

> Aiven puede **apagarse por inactividad**. Síntoma: DNS del host → `NXDOMAIN` y todo lo que toca MySQL responde 503 mientras `/health` sigue en 200 (§15).

---

7. Seguridad

Mecanismo	Implementación
Contraseñas	Hash **BCrypt** con 12 rondas (`BCrypt.Net.BCrypt` en C#, `bcrypt` en Node). Nunca se guarda ni transmite la contraseña en claro.
Sesión	JWT **HS256**, vigencia **15 minutos**. El mismo `JWT_SECRET` firma en login y API.
Bloqueo de cuenta	Tras **5 intentos fallidos** la cuenta se bloquea **15 minutos** (`intentos_fallidos`, `bloqueado_hasta`). Un acceso correcto pone ambos a cero.
Tiempo de respuesta uniforme	Si el usuario no existe se compara contra un **hash señuelo**, para que el tiempo de respuesta no revele qué cuentas están dadas de alta.
Rate limiting	Ventana de 15 minutos sobre intentos de login y de registro.
Tokens de un solo uso	`token_cuenta` guarda solo el **SHA-256** del token (`hash_token`), con `expira_en` y `usado_en`. Consumirlo lo marca como usado dentro de una transacción con `FOR UPDATE`, así no puede reutilizarse.
Antifuerza bruta de carrito	El descuento de stock se hace con `UPDATE … WHERE stock>=@qty`: si no alcanza, no se descuenta nada (no queda stock negativo).
Idempotencia de compra	`venta.clave_idempotencia` evita cobrar dos veces si el cliente reintenta el checkout.
CORS	Orígenes configurables por `CORS_ORIGIN`, pero `.web.app`, `.firebaseapp.com` y `localhost:5173` **se suman siempre en código**, así un olvido en Render no rompe el sitio.
Datos sensibles	`.env` está en `.gitignore`; los repositorios son públicos, por lo que ninguna credencial puede versionarse. La configuración de correo vive en la BD (§8).

---

8. Correo (SMTP)

8.1 De dónde sale la configuración

`Services/MailService.cs` busca las claves **primero en la tabla `config_sistema`** (base de datos en la nube) y, si faltan ahí, en las variables del proceso (`.env` en local, *Environment* en Render). Así se cambia el correo **sin volver a tocar el despliegue**.

```
SELECT clave, valor FROM config_sistema;

-- SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM
```

Si la lectura falla, se reintenta en el siguiente envío (no se queda sin opción).

8.2 Por qué no funciona con Gmail en Render

Render **bloquea el correo saliente en su plan gratuito**. Anuncio oficial:

> *"free Render web services will block outbound network traffic to SMTP ports 25, 465, and 587"*

Evidencia medida desde el propio servidor con `/api/core/correo/diagnostico`:

Destino	Puerto	Resultado
`smtp.gmail.com`	25 / 465 / 587	❌ timeout en IPv4 e IPv6
`www.google.com`	443	✅ 6 ms (la red saliente funciona)
`smtp-relay.brevo.com`	**2525**	✅ 13 ms, EHLO + STARTTLS OK
`smtp.sendgrid.net`	**2525**	✅ 1–10 ms, EHLO + STARTTLS OK
`smtp.mailgun.org`	**2525**	✅ 13 ms, EHLO + STARTTLS OK

Conclusión: solo los puertos de correo están bloqueados; el **2525 no** y es el puerto alternativo que ofrecen los proveedores transaccionales precisamente para este caso.

8.3 Configurar un proveedor (paso a paso)

1. Crear una cuenta gratuita (p. ej. **Brevo**, 300 correos/día sin tarjeta).
2. Verificar el remitente en *Settings* → *Senders*.

3. Generar la clave SMTP en *Settings → SMTP & API → SMTP*.
4. Actualizar la tabla en Aiven:

```
UPDATE config_sistema SET valor='smtp-relay.brevo.com' WHERE clave='SMTP_HOST';

UPDATE config_sistema SET valor='2525'                 WHERE clave='SMTP_PORT';

UPDATE config_sistema SET valor='<login>'              WHERE clave='SMTP_USER';

UPDATE config_sistema SET valor='<clave-smtp>'         WHERE
clave='SMTP_PASSWORD';

UPDATE config_sistema SET valor='Librería Cartagena <<remitente-verificado>>'

  WHERE clave='SMTP_FROM';
```

5. Verificar (no requiere redeploy):

```
curl -X POST
https://libreriacartaagena.onrender.com/api/cuenta/recuperacion/solicitar \

  -H "Content-Type: application/json" -d '{"correo":"tu-correo@gmail.com"}'
```

Respuesta esperada: **200** con *«Si el correo existe, enviaremos instrucciones»*.

8.4 Flujo de recuperación de contraseña

1. `POST /api/cuenta/recuperacion/solicitar` recibe correo **o nombre de usuario**.
2. La API busca la cuenta (`WHERE correo=@dato OR nombre_usuario=@dato`) y **envía al `correo` registrado de esa cuenta** — nunca a una dirección fija. Si no existe la cuenta responde igual (200) para no revelar quién está registrado; en ese caso no se envía nada.
3. Se genera un token de 32 bytes en hexadecimal (64 caracteres), se guarda **hasheado** en `token_cuenta` con vigencia de **20 minutos** y se envía por correo.
4. `POST /api/cuenta/recuperacion/restablecer` valida el token (una sola vez), hashea la contraseña nueva con BCrypt y limpia `intentos_fallidos`.

Si el envío falla, la API responde **503** con el código `SMTP_NO_CONFIGURADO_O_ERROR` y registra el token en el log, en lugar de fingir un éxito.

---

9. Registro, diagnóstico y monitoreo

Endpoint	Uso
`GET /health`	Health check de Render: hace `SELECT 1` y responde `{"status":"ok"}`. **No** depende del SMTP.
`GET /api/core/correo/diagnostico?host=&puerto=`	Diagnostica la salida SMTP **sin enviar ningún correo**: resuelve el DNS, prueba TCP contra cada dirección (IPv4 e IPv6, tope de 8 s c/u) y hace `EHLO` + `STARTTLS`. Es la herramienta con la que se detectó el bloqueo de §8.2.
`GET /scalar/v1`	Documentación interactiva de la API.

Logs locales: `.dev-logs/api.log`, `.dev-logs/login.log`, `.dev-logs/vite.log`.

Mensajes útiles que deja `MailService`:

- `Configuración SMTP leída de config_sistema (5 claves)` → la configuración salió de la nube.
- `No se pudo leer config_sistema; se usan las variables del entorno` → la BD no respondió; se usará el `.env`.
- `Correo NO enviado: faltan claves SMTP …` → falta alguna clave.
- `No se pudo enviar el correo` → el proveedor rechazó o expiró (timeout).

En producción, si algo falla, reproducir primero el diagnóstico: si `tcp` da `TimedOut` en los puertos clásicos y `ok` en 2525, el problema es el bloqueo de Render y no el código.

---

10. reCAPTCHA

- Widget **reCAPTCHA v2 (checkbox)** con `hl=es`, montado en `Frontend/src/components/auth/Recaptcha.tsx`.
- La site key actual es la de **prueba** de Google (`6LeIxAcTAAAAAJcZVRqyHh71UMIEGNQ_MXjiZKhI`): el widget muestra una franja roja de aviso y la clase `.auth-captcha__marco--prueba` recorta el marco (height 55px / translateY −20px) para que no rompa el diseño.
- **Al poner la site key real** de un dominio verificado, esa clase deja de aplicarse automáticamente (`esClaveDePrueba` es falsa) y el widget se dibuja con su tamaño estándar.

No hay que tocar el CSS.

- La secreta (`RECAPTCHA_SECRET`) aún no está configurada en el servidor; sin ella el backend no valida el token, solo el frontend exige que exista.

---

11. Reglas de negocio

Regla	Detalle
Roles	`ADMIN` ve todo; `CLIENTE` compra y consulta; `VENDEDOR` **no tiene panel**; `REPARTIDOR` se usa desde logística.
Alta de usuario	El registro siempre crea cuentas `CLIENTE`; los demás roles los asigna un ADMIN.
Stock	Se descuenta de forma atómica en el checkout (`stock>=@qty`) y se repone en devoluciones y en la recepción de pedidos de compra. `stock_minimo` / `stock_maximo` alimentan las alertas del panel.
Descuentos	`libro.descuento_porcentaje` se aplica al precio y se muestra tachado en el catálogo.
Carrito	Se sincroniza con el servidor por usuario; permite cambiar cantidades, quitar líneas y ver subtotal, envío, descuentos y total.
Compra	El checkout genera `venta` + `detalle_venta`, descuenta stock, registra el domicilio y el envío, y permite descargar el **ticket en PDF** (`PdfService`).
Reseñas	Solo quien compró el libro puede comentar; se muestra el sello «Comprador verificado».
Proveedor	No se puede eliminar si tiene compras registradas (409).
Recuperación de contraseña	Token de un solo uso, 20 minutos, enviado al correo registrado.

---

12. Despliegues

12.1 Frontend → Firebase Hosting

```
cd Frontend

npm run build

firebase deploy --only hosting --project libreria-carta-agena
```

`firebase.json`:

- **Rewrite** `**` → `/index.html` (SPA).
- **Cabeceras** (la última regla que coincide gana): | Origen | Cache-Control | |---|---| | `**` | `no-cache` (los documentos HTML nunca quedan obsoletos) | | `**/*.@(js\|css)` | `public, max-age=3600` | | `/assets/**` | `public, max-age=31536000, immutable` |

> ⚠️ Antes solo `/index.html` tenía `no-cache`; Firebase **no** aplica esa regla a las rutas SPA, así que el navegador guardaba el HTML hasta 1 h y, tras cada despliegue, pedía un bundle ya borrado → **página en blanco** con el error *"Failed to load module script … MIME type text/html"*. Con `** → no-cache` el navegador siempre revalida (ETag 304) y el problema desaparece. Si vuelve a verse en blanco: **Ctrl + F5**.

12.2 Backend → Render

1. Sincronizar `Backend/Cartagena` al repo `Yami678/LibreriaCartaAgena` (§2).
2. Push a `main` → Render construye la imagen con el `Dockerfile` y la publica.
3. Health check: `GET /health` → `{"status":"ok"}`.

Verificación posterior (debe responder todo):

```
B=https://libreriacartaagena.onrender.com

curl -s $B/health

curl -s "$B/api/libros?pagina=1" | head -c 200

curl -s -o /dev/null -w "%{http_code}\n" -X POST $B/api/v1/auth/login \
```

```
  -H "Content-Type: application/json" -d
'{"nombreUsuario":"admin","contrasena":"ClaveDemo123"}'

curl -s -D - -o /dev/null "$B/api/libros" -H "Origin: https://libreria-carta-
agena.firebaseapp.com" \

  | grep -i access-control-allow-origin

curl -s -X POST $B/api/cuenta/recuperacion/solicitar \

  -H "Content-Type: application/json" -d '{"correo":"tu-correo@gmail.com"}'
```

12.3 Credenciales demo

Usuario	Contraseña	Rol
`admin`	`ClaveDemo123`	ADMIN
`cliente`	`ClaveDemo123`	CLIENTE
`vendedor`	`ClaveDemo123`	VENDEDOR (sin panel)
`sgarcia1859`	`NuevaClave123!`	ADMIN (cuenta del equipo)

> Los correos de `admin`, `cliente`, `vendedor` e `inactivo` son **de ejemplo** (`admin@ventasonline.com`, …): el flujo de correo no llega a nadie con ellos.

---

13. Desarrollo local

```
./start-dev.sh      # MySQL + login + API C# + Vite (nativo)

./start-docker.sh   # stack completo con Docker

./stop-dev.sh       # detener todo
```

Servicio	URL
Frontend	http://localhost:5173

Login	http://localhost:3000/api/v1
API C#	http://localhost:5000/api
Scalar	http://localhost:5000/scalar/v1
MySQL	127.0.0.1:3307 / `tienda_online`

```
# Compilar solo la API

dotnet build Backend/Cartagena/backend/Cartagena.Api/Cartagena.Api.csproj



# Probar el contenedor completo contra Aiven (puerto de prueba)

cd Backend/Cartagena/backend/Cartagena.Api && docker build -t cartagena-api-login:local .

docker run --rm -p 18080:8080 \

  -e DB_HOST=… -e DB_PORT=20568 -e DB_NAME=tienda_online -e DB_USER=… \

  -e DB_PASSWORD=… -e JWT_SECRET=… -e JWT_ISSUER=microservicio-login \

  -e CORS_ORIGIN=https://libreria-carta-agena.web.app cartagena-api-login:local
```

13.1 Proxy de Vite (importante)

`Frontend/vite.config.ts` enruta en desarrollo:

Ruta	Destino
`/api/v1`	`http://localhost:3000` (microservicio de login)
`/api`	`http://localhost:5000` (API C#)

> ⚠️ Existía una tercera regla que mandaba `/api/proveedores` a `http://localhost:5001` (el Flask legado, que ya no corre). Eso producía **«Error 500 al consultar /proveedores»** en local mientras en producción funcionaba, porque allí la llamada va directo a Render. La regla se

> eliminó: `/api/proveedores` lo sirve `ProveedoresController.cs` en la API C#. Si vuelve el error, revisa primero `vite.config.ts`.

---

14. Pruebas

`PRUEBAS-ESTRUCTURALES.md` (raíz del repo): **45/45 PASS**, cubriendo autenticación y seguridad, flujo de compra, inventario, logística, panel administrativo y UI (más Lighthouse: accesibilidad 96 %, buenas prácticas 100 %).

Pruebas unitarias del login: `npm test` dentro de `servicios/microservicio-login` (Node test runner + supertest).

---

15. Solución de problemas

Síntoma	Causa	Solución
Página en blanco + *"Failed to load module script"*	HTML cacheado apuntando a un bundle borrado	Ctrl + F5. Causa raíz corregida en `firebase.json` (`** → no-cache`).
`/api/v1/*` responde 404 con errores de Express	El Node está capturando todo el tráfico (puerto mal expuesto)	Solo `EXPOSE 8080`; la API C# debe ser la entrada.
`/api/v1/*` → `502` + *Connection refused (localhost:3001)*	`LOGIN_PROXY_TARGET` apunta al 3001 y el login corre en 3000	Define la variable o habla por Vite (§13.1).
`Error 500 al consultar /proveedores` (solo local)	Vite enrutaba a un Flask retirado en el 5001	Regla eliminada de `vite.config.ts` (§13.1).
`/api/proveedores` → 404	Despliegue anterior sin `ProveedoresController`	Redesplegar (§12.2).
403 en proveedores	Token de un rol distinto a `ADMIN`	Usar cuenta ADMIN (§12.3).
401 en todo	`JWT_SECRET` distinto entre login y	Ambos leen el mismo `.env`;

	API	verificar en Render.
`RUTA_NO_ENCONTRADA` en `/health`	Estás hablando al Express, no al C#	Revisar puertos/`EXPOSE` del `Dockerfile`.
Catálogo/login → 503, `/health` → 200	**Aiven apagado**: el DNS devuelve `NXDOMAIN`	Encender el servicio en la consola de Aiven.
503 `SMTP_NO_CONFIGURADO_O_ERROR` a los 30 s	Puerto de correo bloqueado o proveedor sin configurar	§8: revisar `config_sistema` y el diagnóstico §9.
`GET /api/core/correo/diagnostico` cuelga o da `TimedOut`	Puerto bloqueado por el proveedor de hosting	Comparar contra el 2525 (§8.2).
Aiven: error `3780` al importar	FKs activas durante el dump	`--init-command="SET FOREIGN_KEY_CHECKS=0"`.
CORS bloqueado desde `.firebaseapp.com`	Origen no listado	`Program.cs` suma siempre `.web.app` y `.firebaseapp.com`; revisar `CORS_ORIGIN`.
Login local: 503	MySQL (`3307`) apagado	`docker start mysql-cartagena`.
Puerto ocupado (3000/5000/5173)	Proceso anterior colgado	`./stop-dev.sh`.
Sesión te saca cada 15 min	`JWT_EXPIRES_IN=15m` (diseñado así)	Aumentar la variable en Render si se requiere.

---

16. Glosario

Término	Significado
Aiven	Proveedor de la base de datos MySQL en la nube.
Firebase Hosting	Sirve el frontend estático (SPA).

Render	Sirve el backend dentro de un contenedor único.
YARP	Proxy inverso que hace de puente entre la API C# y el login Node.
Scalar	Generador de documentación interactiva a partir de OpenAPI.
BCrypt	Algoritmo lento y con sal, diseñado para guardar contraseñas.
JWT	Token firmado que transporta el usuario y su rol.
SMTP / 2525	Protocolo de envío de correo; el 2525 es el puerto alternativo que no bloquea Render.
Idempotencia	Que repetir la misma operación no produzca un efecto doble.
